

System Architecture

End-to-end design, Student 1 → Student 4

Full End-to-End System Design (Student 1 → Student 4)

Single source-of-truth design document for the project. Part 1 records the project requirements and scope; Parts 2–9 are the engineering blueprint, role by role.

Document map

- **Part 1** records the full project requirements and scope (problem statement, team roles, model, data strategy, deliverables).
 - **Part 2** explains *the reason* the scattering + attention approach was chosen; it justifies every later decision.
 - **Parts 3–7** are the architecture, role by role (Student 1 → Student 4).
 - **Part 8** is the incremental build order.
 - **Part 9** is the "accuracy is low — what now" playbook (solve problems slowly to reach the best accuracy).
-

PART 1 — PROJECT REQUIREMENTS & SCOPE

****Project.**** Build an Explainable-AI Digital Twin for weld inspection from Phased Array Ultrasonic Testing (PAUT) data, following the problem statement in *"Explainable AI-Based Digital Twin for PAUT Weld Inspection"*, plus the specific methodology defined for this project. The system must (1) automatically detect and classify weld defects from PAUT TFM images, (2) characterize each defect (size, shape, orientation, location, severity), (3) explain its predictions with Explainable AI chosen specifically for this model, and (4) present everything in a Digital Twin of weld health with an interactive dashboard.

****Team & roles (8–10 week internship, 3–4 students).**** Student 1: PAUT data acquisition, preprocessing, signal/image analysis, defect annotation, dataset preparation — **already done; porosity and slag images collected.** Student 2: AI/ML model for defect detection + classification + characterization — **must follow the supervisor's modified approach below, not the basic CNN/YOLO of the PDF.** Student 3: Explainable AI — **must select the single best XAI method(s) for this particular model* from at least {Grad-CAM, Grad-CAM++/CAMs, SHAP, LIME, Guided Backpropagation, Occlusion saliency, SmoothGrad, Integrated Gradients}, justified against the model's architecture and accuracy, adhering to standards.* Student 4: Digital Twin dashboard, weld-health visualization, integration — **must be a research-grade digital twin*, informed by reading digital-twin research papers (what it is, how it is used in science and industry).*

****Defect classes.**** cracks, porosity, slag inclusion, lack of fusion (Student 1 currently has porosity + slag; design must extend to the rest).

****Model (Student 2) — required architecture.**** Use an ****SCN-assisted Attention U-Net**** (improved Wavelet Scattering Network + Attention U-Net), NOT a plain U-Net. Two branches from the input TFM image ($1 \times H \times W$): (a) an Attention U-Net branch that learns the spatial mask, producing encoder features $E1-E4$ + bottleneck; (b) a Wavelet Scattering branch (SCN) producing fixed multiscale features $S1-S4$ + a global scattering vector. ****Fuse at every scale:**** $\text{concat}(E_i, S_i) \rightarrow 1 \times 1 \text{ Conv} \rightarrow \text{SE/CBAM}$ (channel attention then spatial attention). Decode with ****Attention-Gate skip connections****. Output ****three heads****: Head 1 segmentation (background / porosity / slag / ...), Head 2 image-level classification (normal / porosity / slag / mixed), Head 3 **optional** size regression (prefer deriving size from the segmentation mask, not direct regression). ****Loss = $0.6 \times \text{Dice-Focal segmentation} + 0.3 \times \text{defect-classification} + 0.1 \times \text{boundary loss}$ **** (then re-check/tune the loss). Layer design target (tune later): Encoder ConvBlock($1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$), Bottleneck($256 \rightarrow 512$); SCN Wavelet Scattering Transform $J=2$ or 3 , $L=6$ or 8 orientations, $\text{order}=2$; Decoder UpConv($512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$); seg output $1 \times 1 \text{ Conv}$, class output global pooling + MLP. Reference scales: $L1 \ 32 \times 256 \times 256$ / $L2 \ 64 \times 128 \times 128$ / $L3 \ 128 \times 64 \times 64$ / $L4 \ 256 \times 32 \times 32$ / bottleneck $512 \times 16 \times 16$, with each S_i resized to the matching encoder scale. Input patch $1 \times 256 \times 256$.

****Characterization (from masks, use blobs).**** For each predicted defect compute Length = major-axis $\times$ pixel-to-mm scale, Height/Width = minor-axis $\times$ scale, Area = mask-pixels $\times$ mm²/pixel, Orientation = PCA angle, Centroid = mask center, equivalent diameter, aspect ratio, and Defect severity = $f(\text{type, size, orientation, location})$. Use ****blob analysis**** for orientation/sizing so the mathematical measurements are accurate. ****Multi-defect handling:**** instance segmentation (connected components / watershed / Mask-RCNN-style instance head) so several defects in one TFM image are handled separately.

*****Few-shot / weakly-supervised pipeline.***** Stage 1: SCN-guided pseudo-label generation — use the current contour method to generate approximate masks/boxes. Stage 2: train the lightweight SCN-Attention U-Net on those. Stage 3: characterization module. Stage 4: multi-defect instance handling.

*****Data strategy for limited data.***** Do not rely only on conventional augmentation; use *****physics-aware augmentation*****: (1) amplitude variation, (2) speckle/noise injection, (3) defect translation & rotation, (4) simulated attenuation, (5) elastic deformation, (6) synthetic defect insertion into clean TFM backgrounds, (7) mixup/cutmix only within physically plausible defect zones. *****Curriculum:***** start with MORE images, train, observe the problems we hit, fix them to reach best accuracy, then *****progressively reduce***** the number of training images to prove the model works on a limited dataset.

*****Color + grayscale requirement.***** Our TFM images are *both* colour and black-and-white. The model must give best accuracy on either, in training and at inference, so that any new image — colour or greyscale — has its defects detected regardless. (Design implication: normalize both to a single amplitude channel before the SCN; see architecture.)

*****XAI requirement (Student 3).***** Pick the XAI method(s) that best align with THIS specific model (segmentation + classification + scattering features + attention gates), meeting standards and preserving accuracy. Justify the choice against the alternatives.

*****Digital Twin requirement (Student 4).***** Build the *best possible* digital twin: a virtual replica of the weld reproducing geometry, defect locations, inspection history, health status and severity, with real-time-style visualization and an integration layer, grounded in digital-twin research practice.

*****Development approach.***** The project is built incrementally in VS Code, one module at a time, following the defined methodology and problem statement (there is a deliberate reason for this approach — see Part 2). Deliverables: documented Git repo (modular code, version control, requirements file, reproducibility), technical report (problem statement, literature review, PAUT methodology, dataset prep, model, XAI, digital-twin architecture, results/metrics, limitations, future scope, with workflow + architecture diagrams), and a final demo/presentation.

*****Reference template.***** Transfer the design philosophy of He et al. 2025, *"Real-time detection of insulator defects based on improved wavelet scattering convolutional network"* (IWSCN: improved wavelet scattering + small CNN, strong in small-sample regimes) from insulators to PAUT welds.

PART 2 — THE REASON (why the supervisor insists on the scattering + attention approach)

The supervisor specified this particular approach for a deliberate reason. Here it is — a single, coherent reason with several faces:

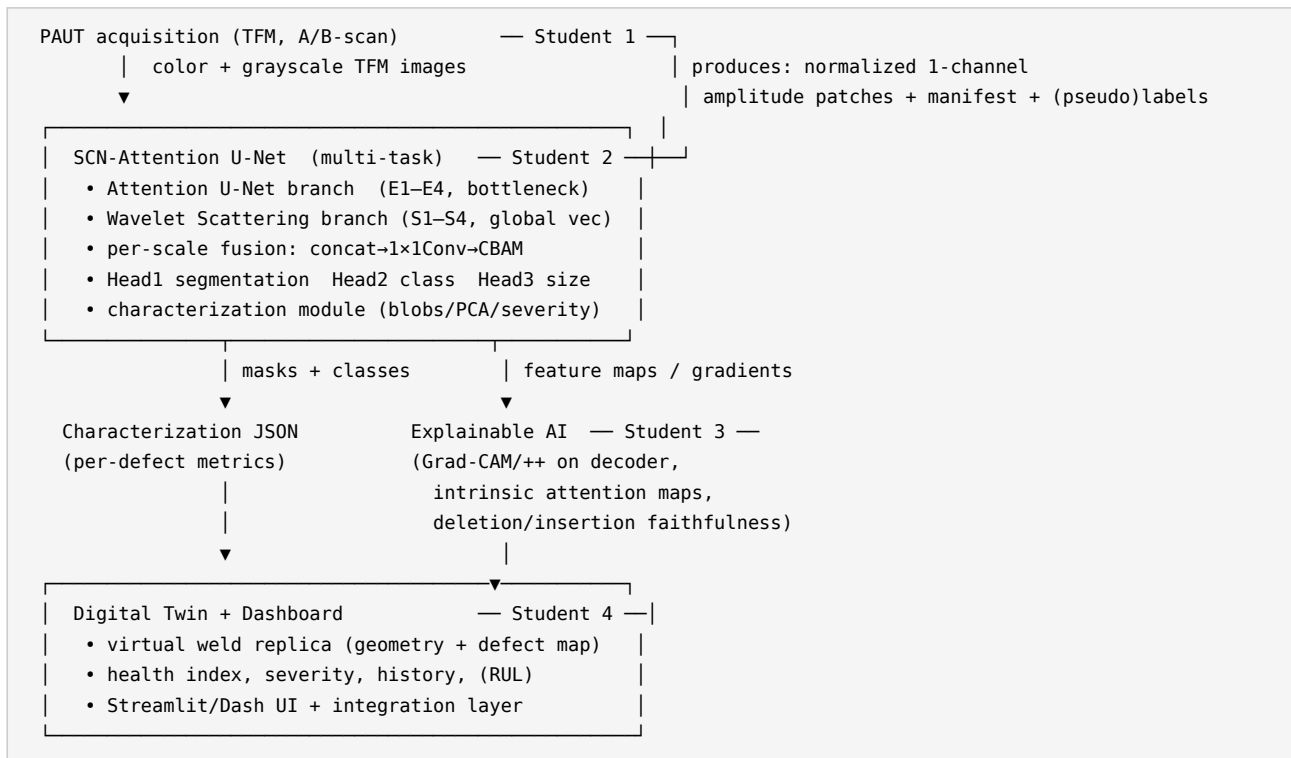
Your real constraint is limited, expensive, noisy data — and the wavelet scattering transform is the single best-known prior for exactly that situation.

1. **Small-sample superiority without training.** A wavelet scattering network uses **fixed** wavelet filters; the scattering stage needs no backpropagation, so it cannot overfit your tiny PAUT dataset. The reference paper and the scattering literature (Bruna & Mallat; Andén & Mallat) show scattering features yield lower error rates than learned CNN features when data is scarce. PAUT weld defects are rare and costly to label — this is precisely the regime where scattering wins.
2. **Built-in invariances that match PAUT physics.** Scattering features are translation-invariant and **stable to deformation** (Lipschitz-continuous to small warps). Weld defects appear at different positions, depths, scales and orientations across TFM images; scattering gives stable features under all of these, so the model generalizes from few examples.
3. **Speckle/noise robustness.** TFM/ultrasonic images carry speckle. The modulus + low-pass averaging in each scattering layer suppresses high-frequency speckle while preserving the coherent defect structure — free denoising baked into the feature extractor.
4. **Sparsity → fewer effective parameters → less overfitting + faster.** Scattering coefficients are sparse (mostly zero outside defect regions), which both reduces compute and gives an interpretable, low-dimensional feature field — helpful for the XAI step too.
5. **Attention U-Net supplies what scattering lacks.** Scattering gives stable **texture/edge** features but not pixel-precise localization or long-range semantic context. The Attention U-Net adds exactly that: precise masks and attention-gated context. Fusing the two means stable small-data features **and** the sharp masks you need to **characterize** defects (size, orientation, severity) and feed the **digital twin**. A plain CNN/YOLO (the PDF's default) would need far more labeled data and would not give you clean masks for characterization.

One-line version for the report: **We adopt an SCN-assisted Attention U-Net because the wavelet scattering prior gives translation-/deformation-invariant, noise-robust features that generalize from very few PAUT samples, while the attention U-Net contributes the precise segmentation required for quantitative defect characterization and digital-twin visualization — solving the small-data problem that defeats conventional CNN/YOLO pipelines.**

PART 3 — SYSTEM ARCHITECTURE OVERVIEW (end-to-end)

Four cooperating layers, one per student, with clean interfaces between them:



Interface contract (write these down — they keep four students unblocked):

- S1 → S2: a folder of single-channel float32 amplitude patches (256×256) + a manifest.csv (image id, source colormap, pixel-to-mm scale, weld id, split) + masks/boxes (pseudo or true).
- S2 → S3: the trained model checkpoint + a function returning, per image, the predicted mask, class logits, the chosen decoder feature maps and the attention-gate coefficients.
- S2 → S4: a defects.json per inspection (list of defect instances with type, centroid_mm, length_mm, width_mm, area_mm2, orientation_deg, severity, confidence).
- S3 → S4: per-defect explanation overlays (PNG heatmaps) + a faithfulness score so the dashboard can show "why" with a trust indicator.

PART 4 — STUDENT 1: DATA, PREPROCESSING, ANNOTATION

Student 1's collection is done, but the *preprocessing decisions here make or break the color/grayscale requirement*, so treat this as active work.

4.1 The color-vs-grayscale decision (most important single choice)

A TFM image is fundamentally a **scalar amplitude field**. "Colour" TFM is just a colormap (e.g., jet/turbo/parula) painted onto that scalar; "grayscale" TFM is the scalar shown directly. So colour carries **no extra information** — it is the same physics. The reference paper confirms the analog: across R/G/B channels their accuracy was identical, so they used a single channel.

Therefore: convert every image to one canonical amplitude channel before anything else.

- Grayscale input → use directly (normalize).
- Colour input → invert the colormap back to a scalar. Practically: if you know the colormap, apply its inverse LUT; if you don't, convert to luminance / or fit nearest-colormap mapping. A robust default is to estimate amplitude by matching each pixel's RGB to the known colormap's lookup table (matplotlib colormaps expose this).
- Then per-image normalize (min-max or z-score on the amplitude) so brightness/gain differences vanish.

This *one* step satisfies "must work on colour and greyscale alike": the SCN never sees colour, only physics-true amplitude. Document it as a design contribution.

4.2 Preprocessing pipeline

1. Load → to canonical amplitude (above).
2. Denoise lightly (median or non-local-means) — keep it gentle; scattering already suppresses speckle.
3. Resize/patch to 256×256 (the input patch size). For large TFM frames, tile into overlapping 256×256 patches and stitch predictions back.
4. Record pixel_to_mm scale per image (from the PAUT setup / probe geometry) — characterization is meaningless without it.
5. Save as float32 .npy (preferred) or 16-bit PNG, plus the manifest.

4.3 Annotation & the Stage-1 pseudo-labels

You likely have few hand labels. Follow the supervisor's Stage-1 plan: generate **pseudo-masks** with the current contour method (threshold on amplitude → morphology → contours → fill → box). These are approximate but enough to bootstrap Stage-2 training. Reserve a small, *carefully hand-labeled* validation set (even 30–50 images) — never train on it; it is your honest accuracy meter.

4.4 Dataset manifest (example columns)

image_id, path, source_type(color/gray), colormap, pixel_to_mm, weld_id, defect_present, label_type(true/pseudo), split(train/val/test)

PART 5 — STUDENT 2: THE SCN-ATTENTION U-NET (the core model)

This is the heart. Build it in PyTorch.

5.1 The scattering branch (SCN)

Use **Kymatio** (Scattering2D) — a maintained wavelet-scattering library, so you don't hand-code wavelets.

- Parameters from the supervisor's sheet: $J = 2$ (start) or 3, $L = 8$ orientations (or 6), $order = 2$.
- For a 256×256 input with $J=2$, $L=8$, $order=2$, scattering returns ~ 81 channels at 64×64 spatial resolution (the field shrinks by 2^J). With $J=3$ you get more channels at 32×32 .
- **No gradients flow into the scattering filters** — they're fixed. This is the small-data advantage; keep it that way.

Producing S1-S4 to match encoder scales (faithful to the supervisor's

table/image 3): Scattering gives one fixed-resolution tensor. To get a feature at each U-Net level, for level i do: $S \rightarrow 1 \times 1$ Conv (to C_i channels) $\rightarrow$ bilinear-resize to $(H_i \times W_i)$. That literally implements $"SCN \text{ coefficients} \rightarrow 1 \times 1 \text{ Conv} \rightarrow \text{resize to encoder scale}."$ The global scattering vector (spatial average) feeds the bottleneck and is also a clean input to the classification head.

5.2 Encoder (Attention U-Net branch)

ConvBlock($1 \rightarrow 32$) $\rightarrow$ ConvBlock($32 \rightarrow 64$) $\rightarrow$ ConvBlock($64 \rightarrow 128$) $\rightarrow$ ConvBlock($128 \rightarrow 256$) $\rightarrow$ Bottleneck($256 \rightarrow 512$), max-pool between levels.

- ConvBlock = (Conv $3 \times 3 \rightarrow$ norm $\rightarrow$ ReLU) $\times 2$. Note the reference paper found *omitting* BatchNorm helped their shallow scattering-CNN; for a U-Net keep norm but consider **GroupNorm** (more stable than BatchNorm with the tiny batch sizes you'll use on limited data).

5.3 Fusion block (per scale) — the SCN $\oplus$ U-Net join

For each level i :

```
fuse_i = Concat(E_i, resize(1x1Conv(S)))    # channels:  $C_i + C_i$ 
fuse_i = 1x1 Conv(2C_i  $\rightarrow$  C_i)           # mix
fuse_i = CBAM(fuse_i)                       # channel attention  $\rightarrow$  spatial attention
```

CBAM (channel + spatial attention) is exactly the "SE/CBAM" the supervisor specified. CBAM is the better pick here because its **spatial** attention map is also a *free, intrinsic explainability signal* for Student 3.

5.4 Decoder with Attention Gates

UpConv($512 \rightarrow 256$) $\rightarrow$ UpConv($256 \rightarrow 128$) $\rightarrow$ UpConv($128 \rightarrow 64$) $\rightarrow$ UpConv($64 \rightarrow 32$), where each skip connection passes through an **Attention Gate** (the gate uses the decoder's gating signal to weight the fused encoder features). Attention gates suppress irrelevant background — important because welds have busy backgrounds and few real defect pixels.

5.5 The three heads

- **Head 1 — Segmentation:** 1×1 Conv $\rightarrow$ C_{classes} channels (background, porosity, slag, crack, lack-of-fusion). Softmax per pixel.
- **Head 2 — Classification:** Global pooling on the bottleneck + global scattering vector $\rightarrow$ MLP $\rightarrow$ {normal, porosity, slag, mixed}. Feeding the scattering vector here directly leverages the small-data strength.
- **Head 3 — Size regression (optional):** keep it optional. Prefer to **derive size from the mask** (more accurate and interpretable). Use Head 3 only as an auxiliary sanity check if at all.

5.6 Loss function (their formula + the re-check the supervisor asked for)

Start with exactly what the supervisor gave:

$$L = 0.6 \cdot L_{\text{DiceFocal}}(\text{seg}) + 0.3 \cdot L_{\text{cls}}(\text{classification}) + 0.1 \cdot L_{\text{boundary}}$$

- **$L_{\text{DiceFocal}}$** = Dice + Focal (handles the heavy background-vs-defect class imbalance; Focal down-weights easy background pixels).
- **L_{cls}** = cross-entropy (or focal CE if classes imbalanced).
- **L_{boundary}** = boundary/contour loss (sharpens mask edges $\rightarrow$ better size/orientation measurements).

The re-check ("you also have to check your loss"): if cracks (thin, elongated) segment poorly, raise the boundary weight (e.g., 0.6/0.25/0.15) and/or add a **Tversky** term ($\beta > 0.5$) that penalizes false negatives — thin defects are mostly false-negative failures. Tune these weights on the validation set; log every change. **Implemented:** a foreground **Focal-Tversky** term ($\alpha=0.3$, $\beta=0.7$) was added per this re-check — it fixed an all-background collapse where the head ignored the rare defect pixels.

5.7 Characterization module (blobs, PCA, severity)

On each predicted instance mask:

- **Blob analysis** (`skimage.measure.regionprops / cv2 blob/contour`) $\rightarrow$ `area_px`, `major_axis`, `minor_axis`, `orientation`, `centroid`, `eccentricity`, `equivalent_diameter`, `solidity`.
- Convert with `pixel_to_mm`: `length_mm = major_axis · s`, `width_mm = minor_axis · s`, `area_mm2 = area_px · s2`, `equiv_diameter_mm`, `aspect_ratio = major/minor`.
- **Orientation** = PCA angle of the mask points (`regionprops` gives this directly; PCA on the (x,y) pixel coordinates is the robust version the supervisor wants).
- **Severity** = $f(\text{type}, \text{size}, \text{orientation}, \text{location})$. A defensible scaffold:

`severity = w_type[type] · norm(size) · orientation_factor · location_factor`, mapped to {minor, moderate, critical}. **Do not invent acceptance thresholds** — calibrate the cut-offs to a real weld-acceptance standard (e.g., ISO 5817 / ASME BPVC) with the supervisor/domain expert. Note this clearly in the report.

5.8 Multi-defect handling

Run **connected-components** on the per-class mask for the simple case; use **watershed** when defects touch; graduate to a **Mask-R-CNN-style instance head** only if needed. Each instance $\rightarrow$ its own characterization record. This is what lets one TFM image report

several defects.

5.9 Physics-aware augmentation (not just rotate/flip)

Implement all seven the supervisor listed, as an albumentations-style pipeline plus custom ops: amplitude variation (gain scaling), speckle/noise injection (multiplicative speckle), defect translation+rotation, simulated attenuation (depth-dependent intensity falloff), elastic deformation, **synthetic defect insertion** (paste a real defect blob into a clean weld background with feathered edges), and **mixup/cutmix restricted to physically plausible defect zones**. These multiply your effective dataset without violating ultrasound physics — the key to limited-data success.

5.10 Training protocol & hyperparameters (starting point — tune)

Setting	Start value	Note
Input patch	1×256×256	single amplitude channel
Optimizer	AdamW	lr 1e-3, weight_decay 1e-4
LR schedule	cosine + 5-epoch warmup	
Batch size	4-8	small data → small batch → use GroupNorm
Epochs	150-200	early-stop on val Dice
Loss weights	0.6 / 0.3 / 0.1	re-tune per §5.6
Scattering	J=2, L=8, order=2	try J=3 if features too coarse
Regularization	dropout 0.1-0.2, heavy aug	combat overfitting
Seed	fixed + logged	reproducibility deliverable

5.11 Limited-data curriculum (exactly as the supervisor described)

1. **Phase A — train on the full set**, fix all problems, push to best accuracy.
2. **Phase B — data ablation**: retrain on 75% → 50% → 25% → 10% of images, plotting val Dice/accuracy vs dataset size. The scattering prior should make this curve **flat** far longer than a plain CNN — that flatness is your headline result proving limited-data robustness. Compare against a plain U-Net baseline to **show** the SCN advantage.

5.12 Evaluation metrics

Segmentation: **Dice / IoU per class**, boundary-F1. Classification: accuracy, precision, recall, F1, confusion matrix (mirror the reference paper's confusion-matrix presentation). Characterization: mean absolute error of size_mm vs ground truth. Always report the **data-ablation curve** from §5.11.

PART 6 — STUDENT 3: EXPLAINABLE AI (chosen for THIS model)

The methodology requires picking the XAI that *aligns with this specific model* — not using everything. Here is the reasoned selection, matched to each part of the architecture.

6.1 Match the method to the head

For the segmentation head → Grad-CAM / Grad-CAM++ (and Seg-Grad-CAM) on the last decoder convolutional block. Evidence: a crack-tip *segmentation* study using a U-Net found gradient-based CAMs (Grad-CAM, Grad-CAM++) significantly more *correct, complete, and compact* than gradient-free CAMs (Score-CAM, Eigen-CAM, Ablation-CAM); Grad-CAM applied to a U-Net's final layer reliably highlights the true target region. Your domain (cracks/porosity/slag) is essentially the same imaging-defect-segmentation setting, so this transfers directly. Grad-CAM also needs **no retraining and no architecture change** and works on any differentiable CNN — zero risk to your accuracy. **Deployed choice: Grad-CAM++** — in our method comparison it was the most faithful explainer (see §6.3), so it is the one wired into the pipeline and dashboard.

For the classification head → Grad-CAM++ for spatial attribution. SHAP was *considered* here for feature attribution (DeepSHAP/GradientSHAP) — it is well-suited to a *classification* CNN/MLP and to the handcrafted shape features feeding severity (it would tell you "porosity area contributed +0.3 toward the 'porosity' class"). **It was not implemented:** the neural classification head is the weak one we deliberately route around (defect *type* comes from the separate scattering classifier, 0.88 balanced), so attributing a head we don't rely on adds a heavy dependency for little value. The implemented XAI is gradient CAMs on the segmentation head plus intrinsic attention maps.

Intrinsic, free explainability → your CBAM spatial-attention maps and Attention-Gate coefficients. Your model *already* produces attention maps. Surface them as a first-class explanation; they require zero extra computation and are faithful by construction.

Secondary / sanity-check methods → Integrated Gradients and SmoothGrad. Use them to corroborate Grad-CAM (agreement across methods = trustworthy explanation). NeuroXAI-style multi-method frameworks do exactly this for U-Net segmentation.

6.2 What to avoid, and why (write this in the report — it shows judgment)

- **SHAP on the segmentation mask:** SHAP is *incompatible with mask-based, multi-channel outputs* (the same reason it fails on Mask-RCNN/YOLO) — so it was never an option for the seg head. It was right *in principle* for the class head, but we did not implement it there either (we route around that head — see §6.1).
- **LIME:** superpixel perturbation is poor for fine speckle/texture in TFM and is unstable run-to-run — not your primary tool.
- **Pure occlusion saliency:** correct but slow and coarse; keep only as an optional cross-check.

6.3 Faithfulness checks (the "standards" the supervisor wants)

Don't just produce pretty heatmaps — *quantify* them: deletion/insertion AUC, and pointing-game accuracy against the ground-truth mask. Report a single trust score per explanation so Student 4's dashboard can display confidence. This is what makes the XAI "adhere to standards." **Result:** the comparison ranked **Grad-CAM++ highest** (trust 0.97, deletion 0.05, insertion 0.95, pointing-game 1.0), ahead of Grad-CAM, LIME and SHAP — so Grad-CAM++ is the deployed explainer.

Selection summary (one table for the report). "Implemented" = what actually runs in `src/xai/`; "Considered" = evaluated in the design but not built.

Model part	Implemented XAI	Considered / not built	Rejected (why)
Segmentation head	Grad-CAM / Grad-CAM++ / Seg-Grad-CAM on last decoder conv	Integrated Gradients	SHAP (no mask support)
Classification head	(head routed around — not explained)	Grad-CAM++, SHAP, SmoothGrad	LIME (unstable on texture)
Fusion/attention	CBAM + Attention-Gate maps (intrinsic)	—	—
Severity features	(reported as engineering measurements)	SHAP on shape features	Occlusion (too slow)
Faithfulness	Deletion/insertion AUC + pointing-game → trust score	—	—

PART 7 — STUDENT 4: DIGITAL TWIN + DASHBOARD

The brief calls for the *best possible* digital twin, grounded in digital-twin research. Here's the design.

7.1 What a digital twin actually is (for the report)

In NDT/Industry-4.0 practice a digital twin is a **virtual replica of a physical asset that reproduces its behaviour under real operating conditions**, used to visualize internal defects, simulate stresses, and anticipate degradation, with AI providing the analytical layer for asset-integrity management. The standard architecture has **three layers**: Physical System → Digital Twin Layer (the virtual model, kept in sync) → Digital Twin Application Layer (dashboards, decisions). Tie it to inspection standards (ISO 9712 operator/method, ISO 9001 quality, and risk-based-inspection / API-580 thinking for severity → action).

7.2 The three layers, concretely for the weld

- **Physical layer:** the welded component + PAUT scan (Student 1's data feed).
- **Twin layer:** a parametric virtual weld — geometry (length, thickness, bead profile), a **defect map** (each defect from defects.json placed at its centroid in weld coordinates, sized/oriented from characterization), **inspection history** (time-stamped scans), a **health index** (e.g., $1 - \sum \text{severity}_i / \text{capacity}$), and optionally **defect-growth / RUL** prediction as a "possible additional module" the PDF mentions.
- **Application layer:** the dashboard below.

7.3 Dashboard (Streamlit or Dash — the PDF's tools)

Recommended **Streamlit** for a beginner (fastest to a working UI). Views:

1. **Weld map** — 2D/3D weld with colour-coded defect markers (severity → colour); click a defect to open its card.
2. **Defect card** — TFM crop + predicted mask overlay + **XAI heatmap** (from Student 3) + measured size/orientation/severity + confidence + trust score.
3. **Health panel** — overall weld health index, defect counts by type, pass/fail vs the chosen acceptance standard.
4. **History/trend** — health index over successive inspections; RUL projection if implemented.
5. **Upload-and-infer** — drop a new TFM image (colour *or* grayscale) → pipeline runs → twin updates live. This visibly demonstrates the colour/grayscale robustness.

7.4 Integration layer

A thin Python service: image → preprocess (S1) → model (S2) → characterization → XAI (S3) → defects.json + overlays → dashboard (S4). Keep it a single callable so the demo is one click. For the "real-time streaming / IoT / cloud" bonus modules, wrap this callable behind a small FastAPI endpoint later — but get the offline pipeline solid first.

PART 8 — BUILD ORDER FOR A BEGINNER (VS Code)

8.1 Environment (do this first)

1. Install Python 3.10+, VS Code, the Python extension, and Git.
2. `python -m venv .venv` → activate it.
3. `pip install torch torchvision kymatio scikit-image opencv-python albumentations numpy pandas matplotlib grad-cam streamlit` → freeze to `requirements.txt`. (shap was considered for the class head but not used — see §6.1; it stays commented out in `requirements.txt`.)
4. `git init`; commit early and often.

8.2 Repository structure (a deliverable in itself)

```
paut-digital-twin/
├─ data/                # (gitignored) raw + processed
├─ src/
│   ├─ data/            # S1: preprocessing, color→amplitude, manifest, pseudo-labels
│   ├─ models/          # S2: scattering branch, attention unet, fusion(CBAM), heads
│   ├─ characterize/    # S2: blobs, PCA, severity
│   ├─ xai/             # S3: gradcam, intrinsic attention, faithfulness metrics
│   ├─ twin/            # S4: weld model, health index, RUL
│   ├─ dashboard/       # S4: streamlit app
│   └─ pipeline.py      # the one-click integration callable
├─ notebooks/           # experiments, ablation curves
├─ configs/             # yaml: hyperparameters (so tuning is logged, not hard-coded)
├─ tests/
├─ requirements.txt
├─ README.md            # setup + reproducibility
└─ report/              # technical report + figures
```

8.3 Suggested 8-10 week milestones

- **W1:** environment + repo + S1 preprocessing (color→amplitude proven on both image types).
- **W2:** pseudo-labels (Stage 1) + dataset manifest + small hand-labeled val set.
- **W3-4:** S2 model — scattering branch, then attention U-Net, then fusion; overfit a tiny subset first to prove it learns.
- **W5:** loss tuning + characterization + multi-defect; reach best accuracy (Phase A).
- **W6:** data-ablation curriculum (Phase B) + baseline comparison.
- **W7:** S3 XAI + faithfulness metrics.
- **W8:** S4 digital twin + dashboard + integration.
- **W9-10:** report, figures, demo, polish, optional RUL/streaming modules.

Beginner tip: at each step, first make it **run on one image**, then make it **correct**, then make it **fast**. Commit after each working step.

PART 9 — "ACCURACY IS LOW" PLAYBOOK (solve slowly, as the supervisor said)

When classification/segmentation underperforms, change **one thing at a time** in this order and log each result:

1. **Data leakage / scale bug first.** Confirm color→amplitude is correct on both image types; confirm val set is never trained on; confirm pixel_to_mm is right (bad scale ruins characterization, not accuracy — but check).
 2. **Can it overfit 10 images?** If not, the model/loss is wrong, not the data. Fix architecture/loss before touching data.
 3. **Class imbalance** (most likely culprit for thin cracks): increase Focal/Tversky emphasis, add boundary-loss weight, oversample defect patches.
 4. **Scattering resolution:** features too coarse → try J=3 and L=8; too heavy → J=2.
 5. **Augmentation:** turn on the physics-aware set; verify synthetic-defect insertion looks realistic (bad synthetics hurt).
 6. **Attention sanity:** view CBAM/attention-gate maps — if they ignore defects, the fusion is mis-wired.
 7. **Then, and only then,** tune lr/schedule/epochs.
 8. **Prove the point:** run the data-ablation curve and the plain-U-Net baseline — if SCN stays flat while the baseline collapses, that **is** your result and your validation of the supervisor's reason.
-

Acceptance-criteria reminder (don't fabricate)

Severity thresholds and pass/fail rules must come from a real weld-acceptance standard (ISO 5817 / ASME BPVC) and your domain expert — not from invented numbers. Flag every such value in the report as standard-derived or as a tunable placeholder pending expert input.

This document is your single source of truth. Build it in the Part-8 order, keep the Part-2 reason in the report's introduction, and let the Part-9 playbook guide you whenever accuracy stalls.